where α is set to a small value like 0.1 in our experiments. The addition of this SFT loss boosts the utilization of positive rollouts and enforces better verification behavior in model training.

Data mixture. At each update, draw a mini-batch $\mathcal{B} = \{x_i\}_{i=1}^{N}$ and split it evenly into two sub-batches $\mathcal{B}_{\text{trace}}$ and $\mathcal{B}_{\text{op}}$ with $|\mathcal{B}_{\text{trace}}| = \lfloor N/2 \rfloor$ and $|\mathcal{B}_{\text{op}}| = \lceil N/2 \rceil$. We train on $\mathcal{B}_{\text{trace}}$ with a standard long-trace objective $\mathcal{J}_{\text{trace}}(\theta)$, and on $\mathcal{B}_{\text{op}}$ with *operator rollouts* under short context, yielding $\mathcal{J}_{\text{op}}(\theta)$. The per-step objective is the average of the two:

$$\mathcal{J}_{\text{train}}(\theta) \;=\; \tfrac{1}{2}\,\mathcal{J}_{\text{trace}}^{\mathcal{B}_{\text{trace}}}(\theta) \;+\; \tfrac{1}{2}\,\mathcal{J}_{\text{op}}^{\mathcal{B}_{\text{op}}}(\theta), \tag{11}$$

where $\mathcal{J}_{\text{trace}}^{\mathcal{B}_{\text{trace}}}$ and $\mathcal{J}_{\text{op}}^{\mathcal{B}_{\text{op}}}$ denote the losses computed on their respective sub-batches. Other ratios are possible; we use a 1:1 split in our experiments.

Mode A: Standard long-trace optimization. Given x, sample a single, long trajectory $s_{1:T} \sim \mathcal{M}_\theta(x)$ and optimize a conventional RL verifiable signal (e.g., a rule based verifiable reward for math problems). This preserves the model's ability to reason in extended traces when available.

Mode B: Operator rollouts under short context. We roll out the same interface used at test time but with one round for stability and cost.

*(i) Parallel-Distill-Refine (**PDR**; one-round rollout).*

1. Generate M parallel generations (reasoning traces, solutions) conditioned on an empty summary:

$$S = \{\, s_i \leftarrow \mathcal{M}_\theta(x, C^{(0)}{=}\varnothing) \,\}_{i=1}^{M}.$$

2. Distill to a bounded, round-wise summary C.
3. Refine a single candidate conditioned on C: $\tilde{s} \leftarrow \mathcal{M}_\theta(x, s_j, C)$.

Why one round during training? Rolling out a single **PDR** round (with M early drafts, distillation to C, and a single refinement) captures the key interface while controlling B_{total} and stabilizing RL. At inference we can run multiple rounds ($R > 1$) using the same operator.

Our datamix preserves competence on long traces while teaching the model to reason across short iterations. **PDR** is emulated by one-round of parallel→distill→refine rollout where the model observes (x, C) and is optimized with a verifiable reward on the final solution trace.

4 Experiments

In this section, we compare the Sequential refinement (**SR**) and Parallel-Distill-Refine (**PDR**) operators against long chain-of-thought baselines under a budget-aware protocol. We measure accuracy with symbolic verifiers like sympy (Meurer et al., 2017) and math-verify[3]. Additionally, we report the results as functions of both the sequential budget B_{seq} (latency proxy along the accepted path) and the total budget B_{total} (all tokens across calls).

We try to answer the following four research questions through our experiments:

- **RQ1:** Can short-context iterations outperform long traces by comparing {**SR**, **PDR**} to long-trace CoT at matched B_{seq} and B_{total}.
- **RQ2:** Figuring out the best distillation strategy for producing $C^{(r)}$ by comparing three $\mathcal{D}$ variants: global summary, extractive top-k, and random-k bootstraps.
- **RQ3:** Identifying the effect of the verification ability of a given model on the final performance.
- **RQ4:** Whether operator-consistent training shifts the Pareto-Frontier. We compare a operator-consistent + standard RL with standard single-trace RL (Sec. 3.3).

[3] https://github.com/huggingface/Math-Verify

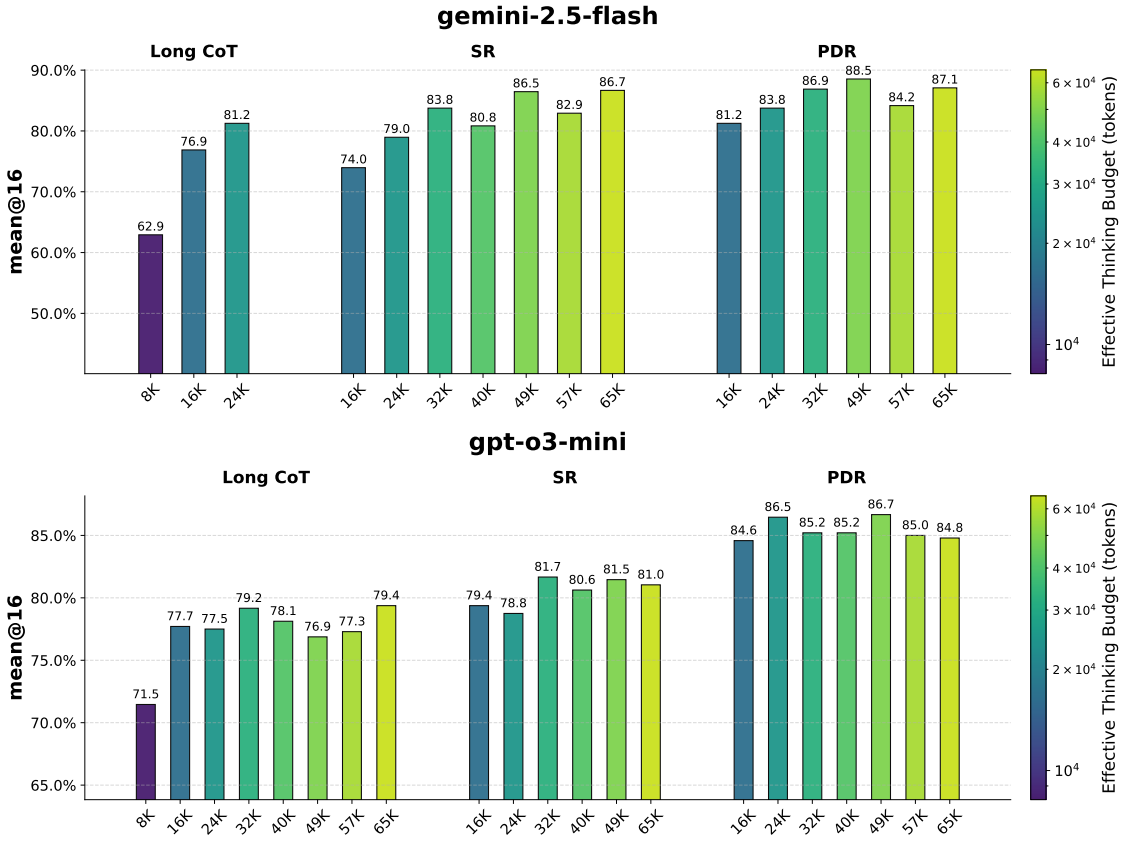

Figure 3 AIME 2024: Iterative improvement beats single-pass long-CoT at matched sequential budgets. The x-axis reports B_{seq}: the thinking tokens consumed along the accepted path of the iterative chain, plus any distilled summary that conditions the next step. Tokens spent on unused parallel proposals are excluded, so B_{seq} serves as a latency proxy. At comparable B_{seq}, both **SR** and **PDR** outperform the single-pass long CoT baseline, with **PDR** yielding the largest gains by converting additional total compute (via parallelism) into accuracy without increasing per-call context.

4.1 Experiments to understand SR and PDR

Setup. We evaluate **SR** and **PDR** as inference-time operators on math problems. Given a prompt x, the model produces a thinking trace and a final solution. The thinking spans, delimited by `<think>` ... `</think>` are stripped out and only the self-contained solutions are used to build the inputs for subsequent rounds. We evaluate on AIME 2024 and AIME 2025 (AoPS, 2025) and report the accuracy computed over 16 independent generations - mean@16.

Models and inference budgets. We evaluate `o3-mini` ("medium" reasoning effort) (OpenAI, 2025) and `gemini-2.5-flash` (Comanici et al., 2025). For `gemini-2.5-flash`, we vary the thinking budget from 8,192 to 24,576 tokens (its maximum), and reserve an additional 2,048 tokens for the final solution. Because `o3-mini` does not expose a separate thinking budget, we vary its maximum generation length from 10,240 to 26,624 tokens to match the same total allowance (assuming 8,192–24,576 thinking tokens plus 2,048 solution tokens). Both operators (**SR** and **PDR**) are compared at matched per-call sequential budgets B_{seq} (latency proxy) while allowing different total token budgets B_{total} via parallelism. All runs use temperature $= 1.0$ and `top-p` $= 1.0$.

RQ1: Do short-context iterations beat long traces at matched latency?

Sequential Refinement (SR). For the **SR** operator, we run `o3-mini` and `gemini-2.5-flash` for thinking budgets $B \in \{8192, 16384, 24576\}$ and refinement rounds $r \in \{1, \ldots, 6\}$. The prompt template is given in §B.1.

SR with a local workspace. We also evaluate a variant of **SR** that inserts a brief, local workspace between refinements: the model first performs *error analysis*: identifying and explaining flaws in the current solution,

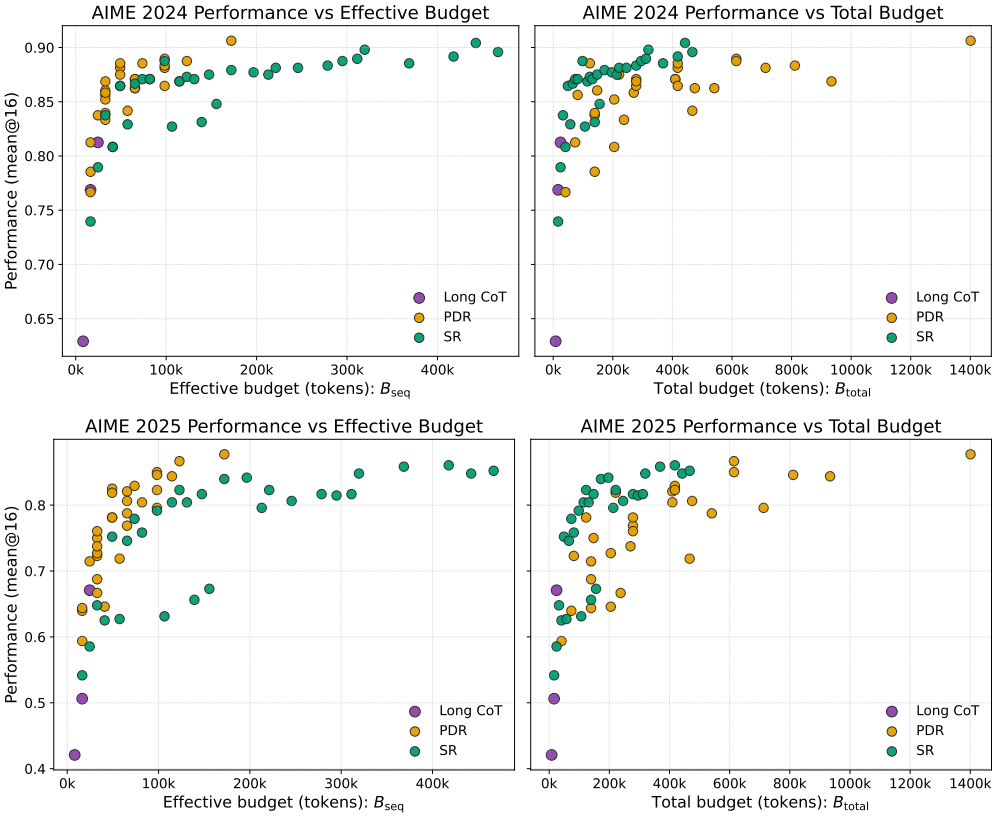

Figure 4 Token Budgets comparison: We plot all the different configurations for Long CoT, **SR** and **PDR** operators for both B_{seq} and B_{total} token budgets for `gemini-2.5-flash`. For B_{seq}, **PDR** forms the Pareto-frontier and gives consistent gains over Long CoT and **SR**. However, for B_{total}, **SR** forms the pareto-frontier because there are no parallel drafts involved so no generations are discarded.

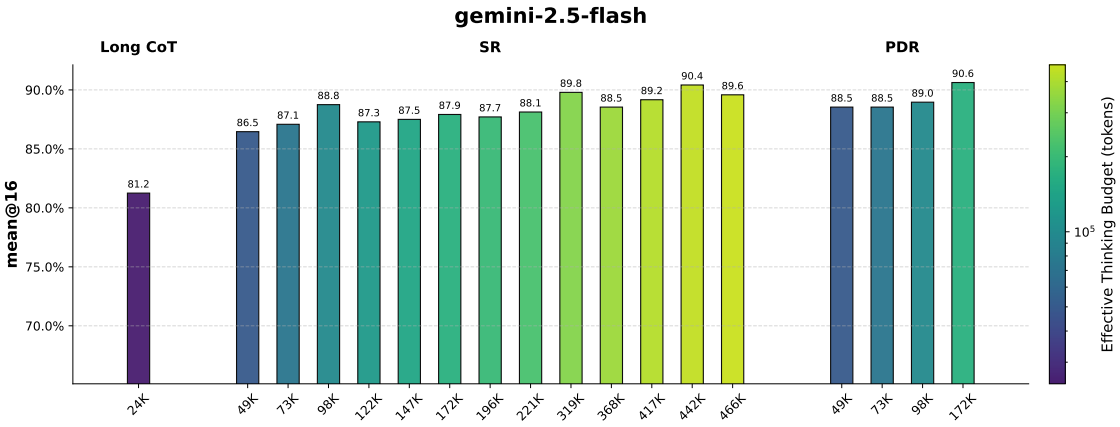

Figure 5 AIME 2024: Long CoT, SR, and PDR at thinking budget of 24576. The x-axis reports B_{seq}: the thinking tokens consumed along the accepted path of the iterative chain, plus any distilled summary that conditions the next step. Tokens spent on unused parallel proposals are excluded, so B_{seq} serves as a latency proxy. At a token matched budget of $442k$ tokens, **SR** has a score of 90.4 but B_{seq} of $442k$, whereas **PDR** has a score of 90.6 but B_{seq} of $172k$ tokens.

and then generates a revised solution conditioned on these notes. All other settings (prompts, budgets) match standard **SR** for a fair comparison. As shown in Table 1, this augmentation is effective for `o3-mini` but not for `gemini-2.5-flash`.